

Inżynieria danych

Wykład 11: odtwarzanie bazy danych po awarii

Katarzyna Grygiel

Semestr letni 2025/2026

Sam commit to za mało

plan

```
T1  
begin  
r(A)  
w(A)  
  
commit
```

bufor



dane na dysku

$A = 1$

Sam commit to za mało

plan

```
T1  
begin  
r(A)  
w(A)  
  
commit
```

bufor

$A = 1$

dane na dysku

$A = 1$

Sam commit to za mało

plan

```
T1  
begin  
r(A)  
w(A)  
  
commit
```

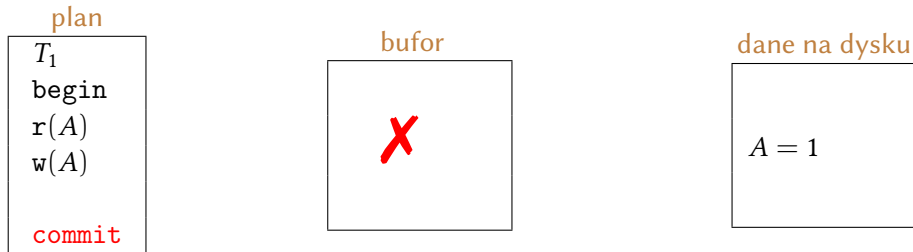
bufor

$A = 2$

dane na dysku

$A = 1$

Sam commit to za mało



Jeśli awaria zdarzy się przed wysłaniem danych z bufora na dysk, to zmiany dokonane przez transakcję zostaną utracone.

Awarie

Istnieje wiele typów awarii, które mogą wystąpić w czasie działania systemu bazodanowego:

- błędy podczas wykonywania transakcji, wśród nich:
 - błędy logiczne (np. naruszenie ograniczeń integralnościowych),
 - błędy systemowe (np. zakleszczenia),

Awarie

Istnieje wiele typów awarii, które mogą wystąpić w czasie działania systemu bazodanowego:

- błędy podczas wykonywania transakcji, wśród nich:
 - błędy logiczne (np. naruszenie ograniczeń integralnościowych),
 - błędy systemowe (np. zakleszczenia),
- awarie systemu:
 - awaria sprzętu (np. wyłączenie zasilania),
 - błąd w oprogramowaniu,

Awarie

Istnieje wiele typów awarii, które mogą wystąpić w czasie działania systemu bazodanowego:

- błędy podczas wykonywania transakcji, wśród nich:
 - błędy logiczne (np. naruszenie ograniczeń integralnościowych),
 - błędy systemowe (np. zakleszczenia),
- awarie systemu:
 - awaria sprzętu (np. wyłączenie zasilania),
 - błąd w oprogramowaniu,
- uszkodzenia dysku.

Awarie

Istnieje wiele typów awarii, które mogą wystąpić w czasie działania systemu bazodanowego:

- błędy podczas wykonywania transakcji, wśród nich:
 - błędy logiczne (np. naruszenie ograniczeń integralnościowych),
 - błędy systemowe (np. zakleszczenia),
- awarie systemu:
 - awaria sprzętu (np. wyłączenie zasilania),
 - błąd w oprogramowaniu,
- uszkodzenia dysku.

System bazodanowy powinien odtworzyć bazę po zaistnieniu awarii pierwszych dwóch typów.

Z trzecim rodzajem awarii można poradzić sobie przez replikację bazy i rozproszenie danych.

Algorytmy odtwarzania bazy po awarii

Integralną częścią systemu zarządzania bazą danych są algorytmy umożliwiające odtworzenie spójnego stanu bazy po awarii systemu.

Algorytmy odtwarzania bazy po awarii

Integralną częścią systemu zarządzania bazą danych są algorytmy umożliwiające odtworzenie spójnego stanu bazy po awarii systemu.

Ponieważ w trakcie odtwarzania bazy danych po awarii serwer jest niedostępny dla użytkowników, czas odtwarzania powinien być zminimalizowany.

Algorytmy odtwarzania bazy po awarii

Integralną częścią systemu zarządzania bazą danych są algorytmy umożliwiające odtworzenie spójnego stanu bazy po awarii systemu.

Ponieważ w trakcie odtwarzania bazy danych po awarii serwer jest niedostępny dla użytkowników, czas odtwarzania powinien być zminimalizowany.

Algorytmy odtwarzania bazy po awarii składają się z dwóch części:

- akcji podejmowanych podczas zwykłego przetwarzania transakcji, które zapewniają zapisywanie wystarczających informacji potrzebnych do odtworzenia stanu bazy sprzed awarii,
- akcji podejmowanych po awarii, prowadzących do powrotu bazy do stanu zapewniającego równocześnie spójność, atomowość i trwałość.

Bufory

Zawartość bazy jest przechowywana przede wszystkim w pamięci nieulotnej (np. na dysku twardym).

Ponieważ tego rodzaju pamięć jest znacznie wolniejsza od pamięci podręcznej, operacje bazodanowe są wykonywane w tej drugiej.

Bufory

Zawartość bazy jest przechowywana przede wszystkim w pamięci nieulotnej (np. na dysku twardym).

Ponieważ tego rodzaju pamięć jest znacznie wolniejsza od pamięci podręcznej, operacje bazodanowe są wykonywane w tej drugiej.

Modyfikowane lub odczytywane dane są kopiowane do bufora pamięci ulotnej, tam następuje operacja zapisu lub odczytu i po zmianach dane przenoszone są z bufora do pamięci trwałej.

Bufory

Zawartość bazy jest przechowywana przede wszystkim w pamięci nieulotnej (np. na dysku twardym).

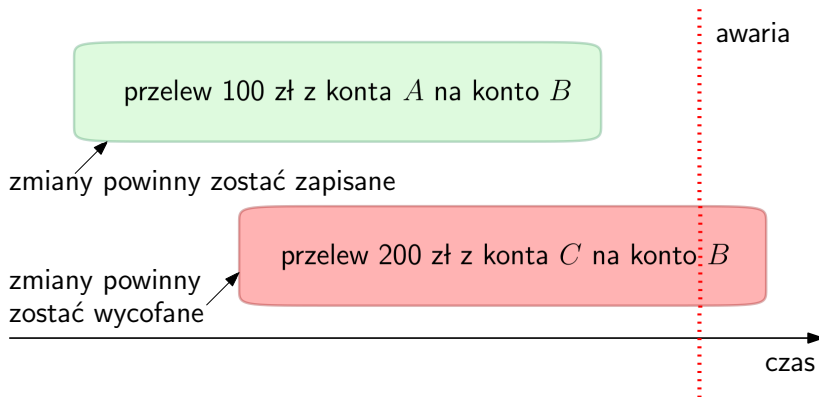
Ponieważ tego rodzaju pamięć jest znacznie wolniejsza od pamięci podręcznej, operacje bazodanowe są wykonywane w tej drugiej.

Modyfikowane lub odczytywane dane są kopiowane do bufora pamięci ulotnej, tam następuje operacja zapisu lub odczytu i po zmianach dane przenoszone są z bufora do pamięci trwałej.

SZDB musi zagwarantować, że:

- zmiany dokonane przez transakcję, które są widoczne dla świata zewnętrznego (transakcja jest widoczna jako zatwierdzona), muszą pozostać trwałe – nie mogą zostać utracone na skutek żadnej z awarii,
- zmiany dokonane przez transakcję wycofaną lub przerwana z powodu awarii nie mogą być trwałe.

Tak to powinno wyglądać



Brudne strony, REDO i UNDO

Strona w buforze jest **brudna**, jeżeli została uaktualniona przez transakcję po czasie jej ostatniego zapisu na dysk.

Brudne strony mogą pozostawać w buforze jeszcze długo po tym, jak uaktualniające je transakcje zostały zaakceptowane.

Jeżeli nastąpiła awaria i aktualizowane strony nie były zapisywane na dysk, to tracimy informacje o dokonanych aktualizacjach.

Brudne strony, REDO i UNDO

Strona w buforze jest **brudna**, jeżeli została uaktualniona przez transakcję po czasie jej ostatniego zapisu na dysk.

Brudne strony mogą pozostawać w buforze jeszcze długo po tym, jak uaktualniające je transakcje zostały zaakceptowane.

Jeżeli nastąpiła awaria i aktualizowane strony nie były zapisywane na dysk, to tracimy informacje o dokonanych aktualizacjach.

W przypadku awarii przeprowadzane są dwa rodzaje operacji.

- **UNDO** to proces wycofywania zmian nieukończonej lub wycofanej transakcji. Dzięki tej operacji mamy zagwarantowaną atomowość.
- **REDO** to proces powtórzenia zatwierdzonej transakcji. Dzięki tej operacji mamy zagwarantowaną trwałość.

Przykład działania bufora i dużo pytań

plan

T_1	T_2
begin	
r(A)	
w(A)	
	begin
	r(B)
	w(B)
	commit
rollback	

bufor



dane na dysku

$A = 1$

$B = 2$

$C = 3$

Przykład działania bufora i dużo pytań

plan

T_1	T_2
begin	
$r(A)$	
$w(A)$	
	begin
	$r(B)$
	$w(B)$
	commit
rollback	

bufor

$A = 1$
$B = 2$
$C = 3$

dane na dysku

$A = 1$
$B = 2$
$C = 3$

Przykład działania bufora i dużo pytań

plan

T_1	T_2
begin	
r(A)	
w(A)	
	begin
	r(B)
	w(B)
	commit
rollback	

bufor

$A = 9$
$B = 2$
$C = 3$

dane na dysku

$A = 1$
$B = 2$
$C = 3$

Przykład działania bufora i dużo pytań

plan

T_1	T_2
begin	
r(A)	
w(A)	
	begin
	r(B)
	w(B)
	commit
rollback	

bufor

$A = 9$

$B = 2$

$C = 3$

dane na dysku

$A = 1$

$B = 2$

$C = 3$

Przykład działania bufora i dużo pytań

plan

T_1	T_2
begin	
r(A)	
w(A)	
	begin
	r(B)
	w(B)
	commit
rollback	

bufor

$A = 9$

$B = 8$

$C = 3$

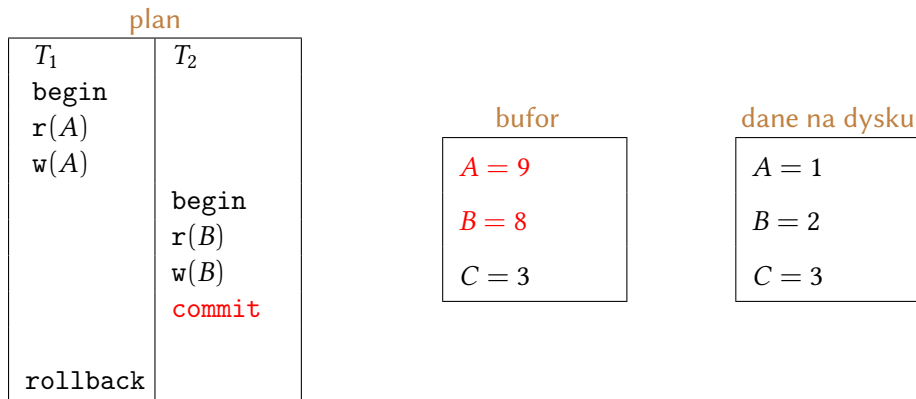
dane na dysku

$A = 1$

$B = 2$

$C = 3$

Przykład działania bufora i dużo pytań



Co się dzieje, gdy zatwierdzamy transakcję T_2 ?

Czy zmiany dokonane przez T_2 muszą zostać zapisane na dysku?

Przykład działania bufora i dużo pytań

plan

T_1	T_2
begin	
r(A)	
w(A)	
	begin
	r(B)
	w(B)
	commit
rollback	

bufor

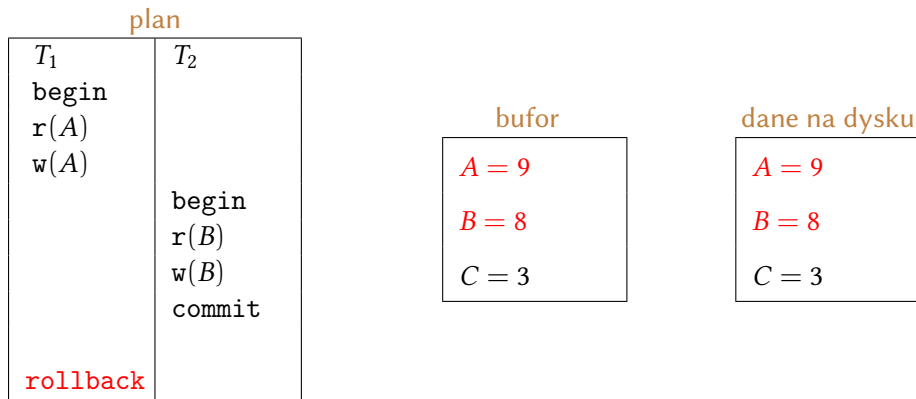
$A = 9$
$B = 8$
$C = 3$

dane na dysku

$A = 9$
$B = 8$
$C = 3$

Czy transakcja T_1 może nadpisać wartość A nawet jeśli nie została zatwierdzona?

Przykład działania bufora i dużo pytań



Co się dzieje, gdy wycofujemy transakcję T_1 ?

(NO-)STEAL i (NO-)FORCE

Czy SZBD pozwala niezatwierdzonej transakcji nadpisać ostatnio zmienioną wartość w pamięci trwałej?

(NO-)STEAL i (NO-)FORCE

Czy SZBD pozwala niezatwierdzonej transakcji nadpisać ostatnio zmienioną wartość w pamięci trwałej?

- strategia **STEAL**: TAK
- strategia **NO-STEAL**: NIE

(NO-)STEAL i (NO-)FORCE

Czy SZBD pozwala niezatwierdzonej transakcji nadpisać ostatnio zmienioną wartość w pamięci trwałej?

- strategia **STEAL**: TAK
- strategia **NO-STEAL**: NIE

Czy SZBD wymaga, aby wszystkie modyfikacje wykonane przez transakcję były odzwierciedlone w pamięci trwałej zanim transakcja może zostać uznana za zatwierdzoną?

(NO-)STEAL i (NO-)FORCE

Czy SZBD pozwala niezatwierdzonej transakcji nadpisać ostatnio zmienioną wartość w pamięci trwałej?

- strategia **STEAL**: TAK
- strategia **NO-STEAL**: NIE

Czy SZBD wymaga, aby wszystkie modyfikacje wykonane przez transakcję były odzwierciedlone w pamięci trwałej zanim transakcja może zostać uznana za zatwierdzoną?

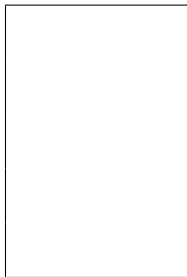
- strategia **FORCE**: TAK
- strategia **NO-FORCE**: NIE

NO-STEAL + FORCE: przykład

plan

T_1	T_2
begin	
r(A)	
w(A)	
	begin
	r(B)
	w(B)
	commit
rollback	

bufor



dysk

$A = 1$

$B = 2$

$C = 3$

NO-STEAL + FORCE: przykład

plan

T_1	T_2
begin	
$r(A)$	
$w(A)$	
	begin
	$r(B)$
	$w(B)$
	commit
rollback	

bufor

$A = 1$
$B = 2$
$C = 3$

dysk

$A = 1$
$B = 2$
$C = 3$

NO-STEAL + FORCE: przykład

plan

T_1	T_2
begin	
r(A)	
w(A)	
	begin
	r(B)
	w(B)
	commit
rollback	

bufor

$A = 17$
$B = 2$
$C = 3$

dysk

$A = 1$
$B = 2$
$C = 3$

NO-STEAL + FORCE: przykład

plan

T_1	T_2
begin	
r(A)	
w(A)	
	begin
	r(B)
	w(B)
	commit
rollback	

bufor

$A = 17$
$B = 2$
$C = 3$

dysk

$A = 1$
$B = 2$
$C = 3$

NO-STEAL + FORCE: przykład

plan

T_1	T_2
begin	
r(A)	
w(A)	
	begin
	r(B)
	$w(B)$
	commit
rollback	

bufor

$A = 17$
$B = 8$
$C = 3$

dysk

$A = 1$
$B = 2$
$C = 3$

NO-STEAL + FORCE: przykład

plan

T_1	T_2
begin	
r(A)	
w(A)	
	begin
	r(B)
	w(B)
	commit
rollback	

bufor

$A = 17$
$B = 8$
$C = 3$
$A = 1$
$B = 8$
$C = 3$

dysk

$A = 1$
$B = 2$
$C = 3$

NO-STEAL + FORCE: przykład

plan

T_1	T_2
begin	
r(A)	
w(A)	
	begin
	r(B)
	w(B)
	commit
rollback	

bufor

$A = 17$
$B = 8$
$C = 3$
$A = 1$
$B = 8$
$C = 3$

dysk

$A = 1$
$B = 8$
$C = 3$

NO-STEAL + FORCE: przykład

plan

T_1	T_2
begin	
r(A)	
w(A)	
	begin
	r(B)
	w(B)
	commit
rollback	

bufor

$A = 17$
$B = 8$
$C = 3$

dysk

$A = 1$
$B = 8$
$C = 3$

NO-STEAL + FORCE

Stosując strategię **NO-STEAL + FORCE** możemy uniknąć wykonywania sekwencji **UNDO** (nie ma zmian do wycofania z dysku) i **REDO** (wszystkie zmiany zapisane na dysku zostały zapisane przez zatwierdzone transakcje).

NO-STEAL + FORCE

Stosując strategię **NO-STEAL + FORCE** możemy uniknąć wykonywania sekwencji **UNDO** (nie ma zmian do wycofania z dysku) i **REDO** (wszystkie zmiany zapisane na dysku zostały zapisane przez zatwierdzone transakcje).

Jest to jednak nieefektywne i może prowadzić do samozakleszczenia transakcji aktywnych ze względu na brak miejsca w buforze dla nowych stron niezbędnych do zakończenia transakcji aktywnych.

Kombinacja strategii **NO-STEAL + FORCE** nie działa dla blokowania i przetwarzania na poziomie rekordów.

NO-STEAL + FORCE

Stosując strategię **NO-STEAL + FORCE** możemy uniknąć wykonywania sekwencji **UNDO** (nie ma zmian do wycofania z dysku) i **REDO** (wszystkie zmiany zapisane na dysku zostały zapisane przez zatwierdzone transakcje).

Jest to jednak nieefektywne i może prowadzić do samozakleszczenia transakcji aktywnych ze względu na brak miejsca w buforze dla nowych stron niezbędnych do zakończenia transakcji aktywnych.

Kombinacja strategii **NO-STEAL + FORCE** nie działa dla blokowania i przetwarzania na poziomie rekordów.

Podstawową kombinacją strategii zarządzania buforem danych jest kombinacja **STEAL + NO-FORCE**.

NO-STEAL + FORCE versus STEAL + NO-FORCE

Szybkość obsługi bufora

	NO-STEAL	STEAL
NO-FORCE	$\times$	szybko
FORCE	wolno	$\times$

NO-STEAL + FORCE versus STEAL + NO-FORCE

Szybkość obsługi bufora

	NO-STEAL	STEAL
NO-FORCE	✗	szybko
FORCE	wolno	✗

Czas odtwarzania bazy po awarii

	NO-STEAL	STEAL
NO-FORCE	✗	wolno
FORCE	szybko	✗

STEAL + NO-FORCE: WAL

Protokół **WAL** (*write-ahead log*) wykorzystuje strategię **STEAL + NO-FORCE**, zapewniając atomowość i trwałość bazy.

STEAL + NO-FORCE: WAL

Protokół **WAL** (*write-ahead log*) wykorzystuje strategię **STEAL + NO-FORCE**, zapewniając atomowość i trwałość bazy.

Poza plikami z danymi przechowywać będziemy plik z **logami** (**dziennik**), w którym zapisywane będą wszystkie zmiany dokonywane przez transakcje.

Zakładamy, że plik z logami:

- znajduje się poza pamięcią ulotną,
- zawiera wystarczające informacje, aby z powodzeniem wykonać operacje **UNDO** i **REDO** w celu odzyskania zawartości bazy.

STEAL + NO-FORCE: WAL

Protokół **WAL** (*write-ahead log*) wykorzystuje strategię **STEAL + NO-FORCE**, zapewniając atomowość i trwałość bazy.

Poza plikami z danymi przechowywać będziemy plik z **logami** (**dziennik**), w którym zapisywane będą wszystkie zmiany dokonywane przez transakcje.

Zakładamy, że plik z logami:

- znajduje się poza pamięcią ulotną,
- zawiera wystarczające informacje, aby z powodzeniem wykonać operacje **UNDO** i **REDO** w celu odzyskania zawartości bazy.

SZBD zapisuje na dysku informacje o zmianach w dzienniku zanim zostaną one wprowadzone na dysku.

Rozpoczęcie i zatwierdzenie każdej transakcji jest odnotowywane w odpowiednim wpisie (`begin` lub `commit`).

Rozpoczęcie i zatwierdzenie każdej transakcji jest odnotowywane w odpowiednim wpisie (`begin` lub `commit`).

Zanim transakcja zostanie uznana za zatwierdzoną (zanim świat zewnętrzny ją taką zobaczy), wszystkie informacje o niej muszą się znaleźć w logu zapisanym na dysku.

Rozpoczęcie i zatwierdzenie każdej transakcji jest odnotowywane w odpowiednim wpisie (`begin` lub `commit`).

Zanim transakcja zostanie uznana za zatwierdzoną (zanim świat zewnętrzny ją taką zobaczy), wszystkie informacje o niej muszą się znaleźć w logu zapisanym na dysku.

Idea: każdy wpis w logu dotyczący zmiany wartości pojedynczego obiektu zawiera następujące informacje:

- identyfikator transakcji,
- identyfikator obiektu,
- wartość przed zmianą (umożliwienie **UNDO**),
- wartość po zmianie (umożliwienie **REDO**).

WAL: przykład działania

plan

```
T1  
begin  
w(A)  
w(B)  
  
commit
```

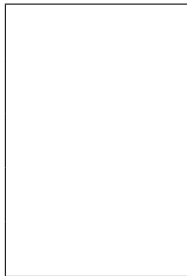
bufor WAL



bufor

```
A = 1  
B = 2  
C = 3
```

log na dysku



dane na dysku

```
A = 1  
B = 2  
C = 3
```

WAL: przykład działania

plan

```
T1  
begin  
w(A)  
w(B)  
  
commit
```

bufor WAL

```
 $\langle T_1, \text{begin} \rangle$ 
```

bufor

```
A = 1  
B = 2  
C = 3
```

log na dysku

dane na dysku

```
A = 1  
B = 2  
C = 3
```

WAL: przykład działania

plan

```
T1  
begin  
w(A)  
w(B)  
  
commit
```

bufor WAL

```
 $\langle T_1, \text{begin} \rangle$   
 $\langle T_1, A, 1, 7 \rangle$ 
```

bufor

```
A = 7  
B = 2  
C = 3
```

log na dysku

dane na dysku

```
A = 1  
B = 2  
C = 3
```

WAL: przykład działania

plan

```
T1  
begin  
w(A)  
w(B)  
  
commit
```

bufor WAL

```
 $\langle T_1, \text{begin} \rangle$   
 $\langle T_1, A, 1, 7 \rangle$   
 $\langle T_1, B, 2, 8 \rangle$ 
```

bufor

```
A = 7  
B = 8  
C = 3
```

log na dysku

dane na dysku

```
A = 1  
B = 2  
C = 3
```

WAL: przykład działania

plan

T_1
begin
 $w(A)$
 $w(B)$

commit

bufor WAL

$\langle T_1, \text{begin} \rangle$
 $\langle T_1, A, 1, 7 \rangle$
 $\langle T_1, B, 2, 8 \rangle$
 $\langle T_1, \text{commit} \rangle$

bufor

$A = 7$
 $B = 8$
 $C = 3$

log na dysku

$\langle T_1, \text{begin} \rangle$
 $\langle T_1, A, 1, 7 \rangle$
 $\langle T_1, B, 2, 8 \rangle$
 $\langle T_1, \text{commit} \rangle$

dane na dysku

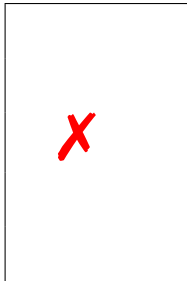
$A = 1$
 $B = 2$
 $C = 3$

WAL: przykład działania

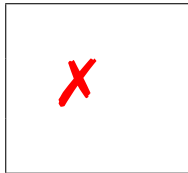
plan

```
T1  
begin  
w(A)  
w(B)  
  
commit
```

bufor WAL



bufor



log na dysku

```
⟨T1, begin⟩  
⟨T1, A, 1, 7⟩  
⟨T1, B, 2, 8⟩  
⟨T1, commit⟩
```

dane na dysku

```
A = 1  
B = 2  
C = 3
```

Kiedy przesyłać logi z dziennika na dysk?

System bazodanowy zapisuje zmiany w logu na dysku:

- gdy transakcja zostanie zatwierdzona,
- można przysyłać jednorazowo zmiany dotyczące kilku transakcji (*group commit*), aby uniknąć narzutu (przesyłanie pojedynczych informacji może spowolnić działanie systemu).

Kiedy przesyłać logi z dziennika na dysk?

System bazodanowy zapisuje zmiany w logu na dysku:

- gdy transakcja zostanie zatwierdzona,
- można przesyłać jednorazowo zmiany dotyczące kilku transakcji (*group commit*), aby uniknąć narzutu (przesyłanie pojedynczych informacji może spowolnić działanie systemu).

Zazwyczaj nie chcemy zapisywać strony na dysk za każdym razem, gdy strona jest aktualizowana przez transakcję.

Strategia **LRU** (*least recently used*) usuwa z bufora stronę najdawniej używaną w ostatnim czasie.

Często aktualizowane strony pozostają w buforze aż zostaną usunięte w wyniku zastosowania strategii LRU lub zostaną zapisane na dysku po ustalonym czasie.

Schematy logowania

- **Schemat fizyczny:** zmiany są zapisywane w sposób niskopoziomowy, dotyczą np. zmian w ciągu bitów.

Ten sposób zapewnia bardzo szybkie wykonywanie operacji REDO i UNDO, jednak zapisy w dzienniku zajmują sporo miejsca.

Schematy logowania

- **Schemat fizyczny:** zmiany są zapisywane w sposób niskopoziomowy, dotyczą np. zmian w ciągu bitów.

Ten sposób zapewnia bardzo szybkie wykonywanie operacji REDO i UNDO, jednak zapisy w dzienniku zajmują sporo miejsca.

- **Schemat logiczny:** zmiany są zapisywane w sposób wysokopoziomowy, np. w postaci pełnych poleceń SQL.

Ten sposób zapewnia krótkie zapisy w dzienniku, jednak czas odtwarzania bazy jest długi.

Schematy logowania

- **Schemat fizyczny:** zmiany są zapisywane w sposób niskopoziomowy, dotyczą np. zmian w ciągu bitów.

Ten sposób zapewnia bardzo szybkie wykonywanie operacji REDO i UNDO, jednak zapisy w dzienniku zajmują sporo miejsca.

- **Schemat logiczny:** zmiany są zapisywane w sposób wysokopoziomowy, np. w postaci pełnych poleceń SQL.

Ten sposób zapewnia krótkie zapisy w dzienniku, jednak czas odtwarzania bazy jest długi.

- **Schemat fizjologiczny:** stanowi kompromis pomiędzy schematami logicznym i fizycznym. Jest najczęściej wykorzystywany w SZBD.

Schematy logowania: przykłady

strona 13

dane	baza
matematyka	
	trójkąt

strona 13

dane	baza
matematyka	
	trójkąt

Schematy logowania: przykłady

strona 13

dane	baza
matematyka	
trujką	

strona 13

dane	baza
matematyka	
trójką	

- schemat fizyczny

strona 13: offset = 345, length = 1,
przed: 'u', po: 'ó'

Schematy logowania: przykłady

strona 13

dane	baza
matematyka	
trójkąt	

strona 13

dane	baza
matematyka	
trójkąt	

- schemat fizyczny

strona 13: offset = 345, length = 1,
przed: 'u', po: 'ó'

- schemat logiczny

```
UPDATE tab SET słowo = 'trójkąt'  
WHERE id = 3;
```

Schematy logowania: przykłady

strona 13

dane	baza
matematyka	
trujką	

strona 13

dane	baza
matematyka	
trójką	

- schemat fizyczny

strona 13: offset = 345, length = 1,
przed: 'u', po: 'ó'

- schemat logiczny

```
UPDATE tab SET słowo = 'trójką'
WHERE id = 3;
```

- schemat fizjologiczny

strona 13: UPDATE(3, 'trujką' -> 'trójką')

Punkty kontrolne

Aby zapobiec rozrastaniu się dzienników w nieskończoność, tworzy się **punkty kontrolne** (*checkpoints*).

Punkty kontrolne

Aby zapobiec rozrastaniu się dzienników w nieskończoność, tworzy się **punkty kontrolne** (*checkpoints*).

Ich rola polega na przesyłaniu do pamięci trwałej wszystkich wpisów z dziennika, które w danym momencie znajdują się w pamięci podręcznej. Po przesłaniu tych wpisów przesyłane są zmiany w bazie (nie tylko informacje o nich).

Punkty kontrolne

Aby zapobiec rozrastaniu się dzienników w nieskończoność, tworzy się punkty kontrolne (*checkpoints*).

Ich rola polega na przesyłaniu do pamięci trwałej wszystkich wpisów z dziennika, które w danym momencie znajdują się w pamięci podręcznej. Po przesłaniu tych wpisów przesyłane są zmiany w bazie (nie tylko informacje o nich).

W przypadku awarii:

- każda transakcja zatwierdzona przed ostatnim punktem kontrolnym jest ignorowana,
- transakcja zatwierdzona później niż ostatni punkt kontrolny zostaje powtórzona (REDO),
- transakcja niezatwierdzona w ogóle zostaje cofnięta (UNDO).

Punkty kontrolne: przykład

WAL

$\langle T_1, \text{begin} \rangle$
 $\langle T_1, A, 1, 71 \rangle$
 $\langle T_1, \text{commit} \rangle$
 $\langle T_2, \text{begin} \rangle$
 $\langle T_2, A, 71, 23 \rangle$
 $\langle T_3, \text{begin} \rangle$
 $\langle \text{checkpoint} \rangle$
 $\langle T_2, \text{commit} \rangle$
 $\langle T_3, A, 23, 78 \rangle$

BUM

Punkty kontrolne: przykład

WAL

$\langle T_1, \text{begin} \rangle$
 $\langle T_1, A, 1, 71 \rangle$
 $\langle T_1, \text{commit} \rangle$
 $\langle T_2, \text{begin} \rangle$
 $\langle T_2, A, 71, 23 \rangle$
 $\langle T_3, \text{begin} \rangle$
 $\langle \text{checkpoint} \rangle$
 $\langle T_2, \text{commit} \rangle$
 $\langle T_3, A, 23, 78 \rangle$

BUM

Transakcja T_1 została zatwierdzona przed punktem kontrolnym, zatem jest ignorowana.

Punkty kontrolne: przykład

WAL

$\langle T_1, \text{begin} \rangle$
 $\langle T_1, A, 1, 71 \rangle$
 $\langle T_1, \text{commit} \rangle$
 $\langle T_2, \text{begin} \rangle$
 $\langle T_2, A, 71, 23 \rangle$
 $\langle T_3, \text{begin} \rangle$
 $\langle \text{checkpoint} \rangle$
 $\langle T_2, \text{commit} \rangle$
 $\langle T_3, A, 23, 78 \rangle$

BUM

Transakcja T_1 została zatwierdzona przed punktem kontrolnym, zatem jest ignorowana.

Transakcje T_2 i T_3 nie zostały zatwierdzone przed ostatnim punktem kontrolnym:

Punkty kontrolne: przykład

WAL

```
⟨T1, begin⟩  
⟨T1, A, 1, 71⟩  
⟨T1, commit⟩  
⟨T2, begin⟩  
⟨T2, A, 71, 23⟩  
⟨T3, begin⟩  
⟨checkpoint⟩  
⟨T2, commit⟩  
⟨T3, A, 23, 78⟩
```

BUM

Transakcja T_1 została zatwierdzona przed punktem kontrolnym, zatem jest ignorowana.

Transakcje T_2 i T_3 nie zostały zatwierdzone przed ostatnim punktem kontrolnym:

- T_2 zostaje powtórzona (REDO),

Punkty kontrolne: przykład

WAL

```
 $\langle T_1, \text{begin} \rangle$   
 $\langle T_1, A, 1, 71 \rangle$   
 $\langle T_1, \text{commit} \rangle$   
 $\langle T_2, \text{begin} \rangle$   
 $\langle T_2, A, 71, 23 \rangle$   
 $\langle T_3, \text{begin} \rangle$   
 $\langle \text{checkpoint} \rangle$   
 $\langle T_2, \text{commit} \rangle$   
 $\langle T_3, A, 23, 78 \rangle$ 
```

BUM

Transakcja T_1 została zatwierdzona przed punktem kontrolnym, zatem jest ignorowana.

Transakcje T_2 i T_3 nie zostały zatwierdzone przed ostatnim punktem kontrolnym:

- T_2 zostaje powtórzona (REDO),
- T_3 zostaje cofnięta (UNDO).

ARIES

Algorytm **ARIES** (*Algorithm for Recovery and Isolation Exploiting Semantics*) to prosta i efektywna metoda odtwarzania transakcji wspierająca blokady i częściowe wycofania, wykorzystująca dziennik tworzony za pomocą protokołu WAL.

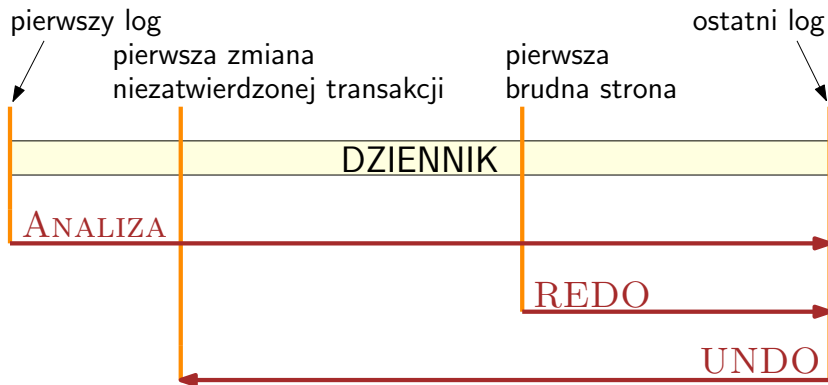
ARIES

Algorytm **ARIES** (*Algorithm for Recovery and Isolation Exploiting Semantics*) to prosta i efektywna metoda odtwarzania transakcji wspierająca blokady i częściowe wycofania, wykorzystująca dziennik tworzony za pomocą protokołu WAL.

Składa się z trzech podstawowych faz:

- analizy dziennika od pierwszego do ostatniego wpisu,
- operacji **REDO** zastosowanej do zmian wprowadzonych od momentu pierwszego możliwego pojawienia się brudnej strony do ostatniego wpisu w dzienniku,
- operacji **UNDO** przebiegającej wstecz od ostatniego wpisu w dzienniku do pierwszej operacji wykonanej przez niezatwierdzoną transakcję.

Punkty startowe faz algorytmu ARIES



Struktury pomocnicze w ARIES

Każdy wpis w dzienniku ma swój unikalny numer **LSN** (*log sequence number*).

Struktury pomocnicze w ARIES

Każdy wpis w dzienniku ma swój unikalny numer **LSN** (*log sequence number*).

Algorytm ARIES korzysta z dwóch dodatkowych tabel:

- **tabeli transakcji** (*TT*), w której przechowywane są identyfikatory niezatwierdzonych transakcji wraz z numerami LSN ostatnich zmian dokonanych przez te transakcje,
- **tabela brudnych stron** (*DPT*), w której przechowywane są numery brudnych stron wraz z numerami LSN pierwszych wpisów, które spowodowały, że strony stały się brudne.

Struktury pomocnicze w ARIES

Każdy wpis w dzienniku ma swój unikalny numer LSN (*log sequence number*).

Algorytm ARIES korzysta z dwóch dodatkowych tabel:

- **tabeli transakcji** (*TT*), w której przechowywane są identyfikatory niezatwierdzonych transakcji wraz z numerami LSN ostatnich zmian dokonanych przez te transakcje,
- **tabela brudnych stron** (*DPT*), w której przechowywane są numery brudnych stron wraz z numerami LSN pierwszych wpisów, które spowodowały, że strony stały się brudne.

<i>TT</i>	
tid	lastLSN
1	34
2	52
7	12

<i>DPT</i>	
pid	recoveryLSN
56	19
29	2
71	15

Zapisy w dzienniku w algorytmie ARIES

Każdy wpis w dzienniku zawiera:

- prevLSN – numer poprzedniego wpisu dotyczącego tej samej transakcji (może być NULL),
- tid – identyfikator transakcji,
- type – typ wpisu.

Zapisy w dzienniku w algorytmie ARIES

Każdy wpis w dzienniku zawiera:

- prevLSN – numer poprzedniego wpisu dotyczącego tej samej transakcji (może być NULL),
- tid – identyfikator transakcji,
- type – typ wpisu.

Jeśli wpis dotyczy modyfikacji (UPDATE), to dodatkowo przechowywane są:

- pid – identyfikator strony,
- redo_info – informacje potrzebne dla operacji REDO,
- undo_info – informacje potrzebne dla operacji UNDO.

Przykładowe wpisy

bufor

strona 13

LSN = —

$A = 71$

$B = 6$

strona 25

LSN = —

$C = 19$

dziennik

tabele

TT

tid lastLSN

DPT

pid recoveryLSN

Przykładowe wpisy

bufor

strona 13

LSN = 1

$A = 72$

$B = 6$

strona 25

LSN = —

$C = 19$

dziennik

1 : $\langle -, 1, U, 13, A = 72, A = 71 \rangle$

tabele

TT

tid	lastLSN
1	1

DPT

pid	recoveryLSN
13	1

Przykładowe wpisy

bufor

strona 13

LSN = 2

$A = 72$

$B = 8$

strona 25

LSN = —

$C = 19$

dziennik

1 : $\langle -, 1, U, 13, A = 72, A = 71 \rangle$

2 : $\langle -, 2, U, 13, B = 8, B = 6 \rangle$

tabele

TT

tid	lastLSN
1	1
2	2

DPT

pid	recoveryLSN
13	1

Przykładowe wpisy

bufor

strona 13

LSN = 2

$A = 72$

$B = 8$

strona 25

LSN = 3

$C = 10$

dziennik

1 : $\langle -, 1, U, 13, A = 72, A = 71 \rangle$

2 : $\langle -, 2, U, 13, B = 8, B = 6 \rangle$

3 : $\langle 2, 2, U, 25, C = 10, C = 19 \rangle$

tabele

TT

tid	lastLSN
1	1
2	3

DPT

pid	recoveryLSN
13	1
25	3

Przykładowe wpisy

bufor

strona 13

LSN = 4

$A = 72$

$B = 10$

strona 25

LSN = 3

$C = 10$

dziennik

1 : $\langle -, 1, U, 13, A = 72, A = 71 \rangle$

2 : $\langle -, 2, U, 13, B = 8, B = 6 \rangle$

3 : $\langle 2, 2, U, 25, C = 10, C = 19 \rangle$

4 : $\langle 1, 1, U, 13, B = 10, B = 8 \rangle$

tabele

TT

tid	lastLSN
1	4
2	3

DPT

pid	recoveryLSN
13	1
25	3

Przykładowe wpisy

bufor

strona 13

LSN = 4

$A = 72$

$B = 10$

strona 25

LSN = 3

$C = 10$

dziennik

1 : $\langle -, 1, U, 13, A = 72, A = 71 \rangle$

2 : $\langle -, 2, U, 13, B = 8, B = 6 \rangle$

3 : $\langle 2, 2, U, 25, C = 10, C = 19 \rangle$

4 : $\langle 1, 1, U, 13, B = 10, B = 8 \rangle$

5 : $\langle 3, 2, \text{commit} \rangle$

tabele

TT

tid	lastLSN
1	4

DPT

pid	recoveryLSN
13	1
25	3

Kompensacja

Aby móc wykonać operację **UNDO**, przechowywane będą również wpisy **kompensacyjne**. Dzięki nim będzie możliwe odtworzenie (**REDO**) zmian cofniętych podczas procesu **UNDO**.

Kompensacja

Aby móc wykonać operację **UNDO**, przechowywane będą również wpisy **kompensacyjne**. Dzięki nim będzie możliwe odtworzenie (**REDO**) zmian cofniętych podczas procesu **UNDO**.

Poza obowiązkowymi wartościami (prevLSN, tid, C [kompensacja]) zawierają dodatkowo:

- pid – numer strony,
- redo_undo_info – informację jak ponownie przeprowadzić zmianę po wykonaniu operacji **UNDO**,
- undo_nextLSN – numer następnego wpisu dotyczącego danej transakcji.

Przykład (cd.)

bufor

strona 13

LSN = 4

$A = 72$

$B = 10$

strona 25

LSN = 3

$C = 10$

dziennik

1 : $\langle -, 1, U, 13, A = 72, A = 71 \rangle$

2 : $\langle -, 2, U, 13, B = 8, B = 6 \rangle$

3 : $\langle 2, 2, U, 25, C = 10, C = 19 \rangle$

4 : $\langle 1, 1, U, 13, B = 10, B = 8 \rangle$

5 : $\langle 3, 2, \text{commit} \rangle$

BUM

tabele

TT

tid	lastLSN
1	4

DPT

pid	recoveryLSN
13	1
25	3

Przykład (cd.)

bufor

strona 13

LSN = 6

$A = 72$

$B = 8$

strona 25

LSN = 3

$C = 10$

dziennik

1 : $\langle -, 1, U, 13, A = 72, A = 71 \rangle$

2 : $\langle -, 2, U, 13, B = 8, B = 6 \rangle$

3 : $\langle 2, 2, U, 25, C = 10, C = 19 \rangle$

4 : $\langle 1, 1, U, 13, B = 10, B = 8 \rangle$

5 : $\langle 3, 2, \text{commit} \rangle$

BUM

6 : $\langle 4, 1, C, 13, B = 8, 1 \rangle$

tabele

<i>TT</i>	
tid	lastLSN
1	6

<i>DPT</i>	
pid	recoveryLSN
13	1
25	3

Przykład (cd.)

bufor

strona 13

LSN = 6

$A = 72$

$B = 8$

strona 25

LSN = 3

$C = 10$

dziennik

1 : $\langle -, 1, U, 13, A = 72, A = 71 \rangle$

2 : $\langle -, 2, U, 13, B = 8, B = 6 \rangle$

3 : $\langle 2, 2, U, 25, C = 10, C = 19 \rangle$

4 : $\langle 1, 1, U, 13, B = 10, B = 8 \rangle$

5 : $\langle 3, 2, \text{commit} \rangle$

BUM

6 : $\langle 4, 1, C, 13, B = 8, 1 \rangle$

BUM

tabele

TT

tid	lastLSN
1	6

DPT

pid	recoveryLSN
13	1
25	3

Przykład (cd.)

bufor

strona 13

LSN = 7

$A = 71$

$B = 8$

strona 25

LSN = 3

$C = 10$

dziennik

1 : $\langle -, 1, U, 13, A = 72, A = 71 \rangle$

2 : $\langle -, 2, U, 13, B = 8, B = 6 \rangle$

3 : $\langle 2, 2, U, 25, C = 10, C = 19 \rangle$

4 : $\langle 1, 1, U, 13, B = 10, B = 8 \rangle$

5 : $\langle 3, 2, \text{commit} \rangle$

BUM

6 : $\langle 4, 1, C, 13, B = 8, 1 \rangle$

BUM

7 : $\langle 6, 1, C, 13, A = 71, - \rangle$

tabele

<i>TT</i>	
tid	lastLSN
1	7

<i>DPT</i>	
pid	recoveryLSN
13	1
25	3

Trzy fazy ARIES raz jeszcze

Analiza:

- rekonstrukcja tabel *DPT* oraz *TT* do stanu sprzed awarii,
- wyznaczenie pierwszego wpisu zawierającego zmiany prowadzące do powstania brudnej strony:

```
SELECT min(recoveryLSN) FROM DPT;
```

Trzy fazy ARIES raz jeszcze

Analiza:

- rekonstrukcja tabel *DPT* oraz *TT* do stanu sprzed awarii,
- wyznaczenie pierwszego wpisu zawierającego zmiany prowadzące do powstania brudnej strony:

```
SELECT min(recoveryLSN) FROM DPT;
```

Faza REDO:

- powtórzenie wszystkich akcji od momentu powstania brudnej strony,
- odtworzenie stanu bazy sprzed awarii (powtórka z historii).

Trzy fazy ARIES raz jeszcze

Analiza:

- rekonstrukcja tabel *DPT* oraz *TT* do stanu sprzed awarii,
- wyznaczenie pierwszego wpisu zawierającego zmiany prowadzące do powstania brudnej strony:

```
SELECT min(recoveryLSN) FROM DPT;
```

Faza REDO:

- powtórzenie wszystkich akcji od momentu powstania brudnej strony,
- odtworzenie stanu bazy sprzed awarii (powtórka z historii).

Faza UNDO:

- cofnięcie wszystkich zmian wprowadzonych przez niezatwierdzone transakcje,
- dodanie wpisów kompensacyjnych w dzienniku.